# UNIT - IV : FILES AND OBJECT-ORIENTED PROGRAMMING

## I. Files

* A FILE is a named location on secondary storage (disk) used to store data permanently. Programs use files to save and read data that must survive after the program ends.

## Types of files

* TEXT files: store characters (ASCII / Unicode) as lines ending with newline '\n'; human readable - .txt, .py, .csv, .html. BINARY files: store data as bytes (0 / 1), not readable in an editor - .jpg, .mp3, .exe, .pdf, .dat, .pkl.

## Opening and closing a file

* open(filename, mode) returns a file object. Always close with close() or use the WITH statement (closes automatically, even on error - best practice).

| Mode | Meaning |
|---|---|
| 'r' | Read (default); file must exist, else FileNotFoundError |
| 'w' | Write; creates new file or ERASES existing content |
| 'a' | Append; adds at the end, creates if not present |
| 'x' | Exclusive create; error if the file exists |
| 'r+' 'w+' 'a+' | Read and write (both) |
| 't' / 'b' | Text (default) / Binary - 'rb', 'wb', 'ab' |

## Creating and reading text data

```
f = open("demo.txt", "w")          # create / write
f.write("Hello Python\n")
f.write("File handling\n")
f.close()

with open("demo.txt", "r") as f:   # read whole file
    data = f.read()
print(data)
```

## File methods to read and write data

| Method | Action |
|---|---|
| read() / read(n) | Whole file / first n characters as a string |
| readline() | Next one line (with \n) |
| readlines() | All lines as a LIST of strings |
| write(str) | Write a string (returns number of chars) |
| writelines(list) | Write a list of strings (no automatic \n) |
| tell() | Current position of file pointer |
| seek(offset, whence) | Move the file pointer; seek(0) = start |
| flush(), close() | Force write buffer / close file |
| name, mode, closed | File attributes |

```
with open("demo.txt") as f:
    print(f.readline(), end="")        # first line
    print(f.tell())                    # pointer position
    rest = f.readlines()               # remaining lines (list)
f = open("demo.txt")
for line in f:                         # memory efficient loop
    print(line.strip())
f.close()
with open("demo.txt", "a") as f:       # append
    f.write("New line added\n")
```

## Reading and writing binary files

```
data = bytes([65, 66, 67, 68])
with open("bin.dat", "wb") as f:
    f.write(data)
with open("bin.dat", "rb") as f:
    print(f.read())                    # b'ABCD'

# copy an image (binary)
with open("a.jpg", "rb") as src, open("b.jpg", "wb") as dst:
    dst.write(src.read())
```

## Pickle module

* PICKLE serialises (pickling) a Python object (list, dict, class object) into a byte stream and stores it in a binary file; unpickling restores the object. pickle.dump(obj, file) and pickle.load(file). Open in 'wb' / 'rb'. Never unpickle data from untrusted sources (security risk).

```
import pickle
student = {"name": "Anu", "marks": [85, 90, 78]}
with open("stu.pkl", "wb") as f:
    pickle.dump(student, f)            # pickling
with open("stu.pkl", "rb") as f:
    obj = pickle.load(f)               # unpickling
print(obj["name"], obj["marks"])
```

## Reading and writing CSV files

* CSV = Comma Separated Values, a plain text table (each line a row, items separated by commas). Use the csv module: csv.reader / csv.writer (lists) and csv.DictReader / csv.DictWriter (dictionaries). Open with newline='' when writing.

```
import csv
rows = [["Name", "Marks"], ["Anu", 85], ["Raj", 72]]
with open("m.csv", "w", newline="") as f:
    w = csv.writer(f)
    w.writerow(["ID", "Name"])         # single row
    w.writerows(rows)                  # many rows
with open("m.csv", "r") as f:
    for row in csv.reader(f):
        print(row)                     # ['Name', '85'] ...
with open("m.csv") as f:
    for r in csv.DictReader(f):        # first row = keys
        print(r["Name"], r["Marks"])
```

## Python os and os.path modules

| Function | Purpose |
|---|---|
| os.getcwd() | Current working directory |
| os.listdir(path) | List files and folders |
| os.mkdir(d), os.makedirs(p) | Create directory / nested directories |
| os.rmdir(d), os.remove(f) | Delete empty directory / file |
| os.rename(old, new) | Rename a file or directory |
| os.chdir(path) | Change directory |
| os.path.exists(p) | True if path exists |
| os.path.isfile(p), isdir(p) | Is it a file / directory? |
| os.path.join(a, b) | Join path parts with correct separator |
| os.path.getsize(p) | Size in bytes |
| os.path.splitext(p) | Split name and extension |
| os.path.basename(p), dirname(p) | File name / folder part |

```
import os
print(os.getcwd())
if not os.path.exists("data"):
    os.mkdir("data")
p = os.path.join("data", "a.txt")
open(p, "w").write("hi")
print(os.path.getsize(p), os.path.splitext(p))   # 2 ('data/a','.txt')
print(os.listdir("data"))                         # ['a.txt']
os.remove(p); os.rmdir("data")
```

# 2. Object-Oriented Programming (OOP)

* OOP organises a program around OBJECTS (data + functions together) instead of functions only. Main concepts: CLASS, OBJECT, ENCAPSULATION, INHERITANCE, POLYMORPHISM, ABSTRACTION. Benefits: reusability, modularity, security (data hiding), easy maintenance, real-world modelling.

## Classes and objects

* CLASS = blueprint / template that defines attributes (variables) and methods (functions). OBJECT = an instance of a class, with its own copy of data. Example: class Car -> objects car1, car2.

## Creating classes and objects in Python

```
class Student:                          # class definition
    college = "JNTU"                    # class attribute
    def show(self):                     # method
        print("I am a student")

s1 = Student()                          # object creation
s1.show()                               # method call
print(s1.college, type(s1))
```

## Constructor method (__init__)

* __init__(self, ...) is a special method CALLED AUTOMATICALLY when an object is created, to initialise data attributes. self refers to the current object (first parameter of every instance method). Python allows only one __init__ (use default arguments for flexibility). __del__ is the destructor.

```
class Student:
    def __init__(self, name, roll):
        self.name = name              # data (instance) attributes
        self.roll = roll
    def display(self):
        print(self.roll, self.name)

s = Student("Anu", 101)
s.display()                           # 101 Anu
```

## Classes with multiple objects

```
s1 = Student("Anu", 101)
s2 = Student("Raj", 102)
s3 = Student("Sam", 103)
for obj in (s1, s2, s3):
    obj.display()                     # each has its own data
print(s1 is s2, id(s1) != id(s2))     # False True
```

## Class attributes vs data (instance) attributes

| Class attribute | Data (instance) attribute |
|---|---|
| Defined in class body, outside methods | Defined inside __init__ / methods with self. |
| SHARED by all objects (one copy) | Separate copy for every object |
| Accessed by ClassName.attr or object.attr | Accessed by object.attr only |
| Changing via class affects all objects | Changing affects only that object |

```
class Employee:
    company = "TCS"                    # class attribute
    count = 0
    def __init__(self, name):
        self.name = name               # instance attribute
        Employee.count += 1
e1 = Employee("A"); e2 = Employee("B")
print(Employee.count, e1.company)      # 2 TCS
Employee.company = "Infosys"
print(e2.company)                      # Infosys (shared)
e1.name = "Z"; print(e2.name)          # B (not shared)
```

## Encapsulation

* Wrapping data and the methods that work on it into ONE unit (class) and restricting direct access to the data = DATA HIDING. Access levels by naming: public (name), protected (_name, convention only), private (__name, name mangled to _Class__name). Use getter / setter methods.

```
class Account:
    def __init__(self, bal):
        self.__balance = bal          # private
    def deposit(self, amt):
        if amt > 0:
            self.__balance += amt
    def get_balance(self):            # getter
        return self.__balance
a = Account(1000)
a.deposit(500)
print(a.get_balance())                # 1500
print(a.__balance)                    # AttributeError
print(a._Account__balance)            # 1500 (name mangling)
```

## Inheritance

* A new class (CHILD / derived / sub class) acquires the attributes and methods of an existing class (PARENT / base / super class) -> code reuse. Syntax: class Child(Parent). super() calls the parent's method / constructor. Method overriding: child redefines a parent method.
* Types: SINGLE (one parent), MULTIPLE (many parents: class C(A, B)), MULTILEVEL (A -> B -> C), HIERARCHICAL (one parent, many children), HYBRID.

```
class Person:                                   # parent
    def __init__(self, name):
        self.name = name
    def show(self):
        print("Name:", self.name)

class Student(Person):                          # single inheritance
    def __init__(self, name, roll):
        super().__init__(name)                  # call parent constructor
        self.roll = roll
    def show(self):                             # method overriding
        super().show()
        print("Roll:", self.roll)

Student("Anu", 7).show()
print(issubclass(Student, Person), isinstance(Student("a",1), Person))
```

```
class A: pass
class B(A): pass            # multilevel: A -> B -> C
class C(B): pass
class D: pass
class E(A, D): pass         # multiple inheritance
print(E.__mro__)            # method resolution order
```

## Polymorphism

* 'Many forms': the same function / method name behaves differently for different objects or inputs. Types: (1) operator overloading (+ on int adds, on str joins; define __add__), (2) METHOD OVERRIDING

(run-time, via inheritance), (3) duck typing (any object with the needed method works). Python has no compile-time method overloading - use default / *args.

```
class Dog:
    def sound(self): print("Bow Bow")
class Cat:
    def sound(self): print("Meow")
for animal in (Dog(), Cat()):     # same call, different result
    animal.sound()

class P:
    def __init__(self, x): self.x = x
    def __add__(self, o):         # operator overloading
        return P(self.x + o.x)
print((P(2) + P(3)).x)            # 5
print(len("abc"), len([1, 2]))    # one function, many types
```

## Special (magic) methods and abstraction

| Method | Called when |
|---|---|
| __init__(self, ...) | object is created |
| __str__(self) | print(obj) / str(obj) |
| __repr__(self) | repr(obj), debugging display |
| __len__(self) | len(obj) |
| __add__, __sub__, __eq__, __lt__ | obj + obj, -, ==, < |
| __del__(self) | object is destroyed |

```
from abc import ABC, abstractmethod
class Vehicle(ABC):                    # abstract class
    @abstractmethod
    def start(self): pass              # must be overridden
class Car(Vehicle):
    def start(self): print("Car starts")
    def __str__(self): return "I am a car"
# v = Vehicle()  -> TypeError (cannot instantiate abstract class)
c = Car(); c.start(); print(c)         # Car starts / I am a car
```

* ABSTRACTION = showing only essential features and hiding implementation details (abstract classes, interfaces). Static method (@staticmethod) needs no self; class method (@classmethod) gets cls - used for alternate constructors.

### Important questions

* 1. Explain file modes and file methods; difference between text and binary files.
* 2. What is pickling? Write a program to store and retrieve an object.
* 3. Explain constructor, self, class attribute vs data attribute with examples.
* 4. Explain types of inheritance with programs; what is method overriding?
* 5. Explain encapsulation and polymorphism in Python.

## 3. Sample Experiments (18 - 23)

### Exp 18: Sort words of a file into another file (lower-case)

```
with open("source.txt") as f:
    words = f.read().lower().split()    # lower-case, split
words.sort()
with open("output.txt", "w") as g:
    for w in words:
        g.write(w + "\n")
print("Sorted words written to output.txt")
# source: "Banana apple Cherry" -> output: apple banana cherry
```

## Exp 19: Print each line of a file in reverse order

```
with open("source.txt") as f:
    for line in f:
        print(line.rstrip("\n")[::-1])     # reverse characters
# to print lines in reverse order instead:
#   for line in reversed(f.readlines()): print(line)
```

## Exp 20: Count characters, words and lines in a file

```
chars = words = lines = 0
with open("source.txt") as f:
    for line in f:
        lines += 1
        words += len(line.split())
        chars += len(line)
print("Lines:", lines, "Words:", words, "Characters:", chars)
```

## Exp 21: Array - create, display, append, insert, reverse

```
from array import array
a = array("i", [10, 20, 30, 40])      # create (typecode i = int)
print("Array:", a)
a.append(50)                          # append
a.insert(2, 25)                       # insert at index 2
print("After append/insert:", a.tolist())
a.reverse()                           # reverse
print("Reversed:", a.tolist())        # [50, 40, 30, 25, 20, 10]
for x in a: print(x, end=" ")         # display
```

## Exp 22: Add, transpose and multiply two matrices

```
A = [[1, 2, 3], [4, 5, 6], [7, 8, 9]]
B = [[9, 8, 7], [6, 5, 4], [3, 2, 1]]
add = [[A[i][j] + B[i][j] for j in range(3)] for i in range(3)]
trans = [[A[j][i] for j in range(3)] for i in range(3)]
mul = [[0]*3 for _ in range(3)]
for i in range(3):
    for j in range(3):
        for k in range(3):
            mul[i][j] += A[i][k] * B[k][j]
print("Sum:", add)       # [[10,10,10],[10,10,10],[10,10,10]]
print("Transpose:", trans)   # [[1,4,7],[2,5,8],[3,6,9]]
print("Product:", mul)   # [[30,24,18],[84,69,54],[138,114,90]]
```

## Exp 23: Shape class with area and perimeter (Circle, Triangle, Square,

```
import math
class Shape:
    def area(self): pass
    def perimeter(self): pass
class Circle(Shape):
    def __init__(self, r): self.r = r
    def area(self): return math.pi * self.r ** 2
    def perimeter(self): return 2 * math.pi * self.r
class Triangle(Shape):
    def __init__(self, a, b, c): self.a, self.b, self.c = a, b, c
    def perimeter(self): return self.a + self.b + self.c
    def area(self):                          # Heron's formula
        s = self.perimeter() / 2
        return math.sqrt(s*(s-self.a)*(s-self.b)*(s-self.c))
class Square(Shape):
    def __init__(self, s): self.s = s
    def area(self): return self.s ** 2
    def perimeter(self): return 4 * self.s
for sh in (Circle(5), Triangle(3, 4, 5), Square(4)):
    print(type(sh).__name__, round(sh.area(), 2), round(sh.perimeter()
# Circle 78.54 31.42 / Triangle 6.0 12 / Square 16 16
```

**Revision: modes r w a x (+ b); with-statement; pickle.dump / load; csv.reader / writer; os.path.exists. OOP: class, object, __init__, self, encapsulation (__private), inheritance (super()), polymorphism (overriding).**